



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа № 2

Студент группы *ИКБО-06-22, Ляхов Т. А.*

(подпись)

Проверил *Старший преподаватель,
Приходько Н.А.*

(подпись)

Отчет представлен «__»_____2025 г.

Москва 2025 г.

Теоретическое введение

Современный анализ данных сталкивается с проблемой многомерности — большинство реальных наборов данных содержат десятки и сотни признаков, что делает их непосредственное восприятие и анализ человеком практически невозможным. Визуализация многомерных данных представляет собой процесс проецирования высокоразмерных данных в пространство меньшей размерности с сохранением ключевых структур и взаимосвязей. Это позволяет преобразовать сложные числовые массивы в интуитивно понятные графические представления, выявляя скрытые закономерности, кластеры и аномалии.

Для работы с многомерными данными в Python существует комплекс библиотек, каждая из которых решает специфические задачи. Библиотека Pandas предоставляет мощные инструменты для работы со структурированными данными, включая загрузку, очистку, преобразование и агрегацию данных. Matplotlib служит основой для создания статических визуализаций, предлагая широкий спектр типов графиков от простых линейных диаграмм до сложных многокомпонентных визуализаций. Plotly расширяет эти возможности, добавляя интерактивность и расширенные функции кастомизации, что особенно ценно для исследовательского анализа данных.

В арсенале визуализации выделяются несколько ключевых типов графиков, каждый из которых оптимален для определенных задач. Столбчатые диаграммы эффективны для сравнения категориальных данных и отслеживания изменений во времени. Круговые диаграммы наглядно представляют пропорциональные соотношения и доли целого, хотя их применение ограничивается небольшим количеством категорий для сохранения читаемости. Линейные графики идеально подходят для анализа тенденций и выявления зависимостей между переменными.

Особую сложность представляет визуализация действительно многомерных данных, где количество признаков исчисляется десятками и

сотнями. Для этой задачи применяются специализированные алгоритмы снижения размерности. Метод t-SNE (t-Distributed Stochastic Neighbor Embedding) использует вероятностный подход к сохранению локальных структур данных, эффективно выделяя кластеры и группы. Более современный алгоритм UMAP (Uniform Manifold Approximation and Projection) основан на теоретических положениях топологии и теории многообразий, что позволяет ему сохранять как локальную, так и глобальную структуру данных с высокой вычислительной эффективностью.

Каждый из этих методов имеет свои параметры настройки, которые существенно влияют на результат визуализации. В t-SNE ключевым параметром является перплексия, определяющая баланс между вниманием к локальным и глобальным структурам данных. В UMAP основными настраиваемыми параметрами выступают количество соседей и минимальное расстояние, которые совместно определяют детализацию и плотность получаемой проекции. Понимание и грамотное использование этих параметров позволяет получать наиболее информативные и интерпретируемые визуализации, что является *crucial* для успешного анализа многомерных данных.

Практическое освоение этих методов и инструментов составляет основу данной работы, направленной на формирование комплексных навыков визуализации и анализа многомерных данных от базовых операций предобработки до сложных методов снижения размерности.

Задание №1

Условие: *найти и выгрузить многомерные данные (с большим количеством признаков – столбцов) с использованием библиотеки pandas. В отчёте описать найденные данные.*

Для работы был выбран [бесплатный датасет о продажах компании “BMW” в период с 2010 по 2024 год](#). Этот набор данных содержит подробную информацию о продажах автомобилей BMW с 2010 по 2024 год в разных регионах мира. Он включает такие атрибуты, как модель, год выпуска, объем двигателя, пробег, тип трансмиссии, тип топлива, цена и объем продаж. Ученые и аналитики могут использовать его для изучения тенденций рынка, подходов к ценообразованию и предпочтений клиентов.

Задание №2

Условие: *вывести информацию о данных при помощи методов .info(), .head(). Проверить данные на наличие пустых значений. В случае их наличия удалить данные строки или интерполировать пропущенные значения. При необходимости дополнительно преобразовать данные для дальнейшей работы с ними.*

Для выполнения задачи первичного анализа данных была разработана программа на Python с использованием библиотек Pandas и NumPy. Решение включает несколько ключевых этапов:

Первым шагом производится настройка параметров отображения Pandas для обеспечения удобного просмотра данных - устанавливается отображение всех столбцов без ограничений, задается ширина вывода и максимальная ширина столбцов.

Далее программа загружает данные из CSV-файла с разделителем-запятой в DataFrame. После загрузки выполняется сброс индекса, что является хорошей практикой для обеспечения корректной нумерации строк.

Основной анализ состоит из трех частей: с помощью метода .info() выводится общая информация о структуре данных, включая типы столбцов и количеств ненулевых значений; метод .head() позволяет просмотреть первые

пять строк набора данных для визуальной оценки содержимого.

О с о б о е внимание уделяется анализу пропущенных значений - программа подсчитывает абсолютное и процентное количество пропусков по каждому столбцу, создавая сводную таблицу. Если пропуски отсутствуют, выводится соответствующее сообщение. В случае обнаружения пропущенных значений, представленная статистика позволяет принять обоснованное решение о стратегии их обработки - удалении строк или интерполяции значений.

Демонстрация работы программы изображена на рисунке 1.

```
C:\Users\timof\AppData\Local\Programs\Python\Python313\python.exe D:\Programming\MIREA_BigData\second_prac\2_info_n_head.py
=== ПЕРВИЧНЫЙ АНАЛИЗ ДАННЫХ BMW ===

1. .info() набора данных:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Model                  50000 non-null  object
1   Year                   50000 non-null  int64
2   Region                 50000 non-null  object
3   Color                  50000 non-null  object
4   Fuel_Type              50000 non-null  object
5   Transmission           50000 non-null  object
6   Engine_Size_L          50000 non-null  float64
7   Mileage_KM             50000 non-null  int64
8   Price_USD              50000 non-null  int64
9   Sales_Volume           50000 non-null  int64
10  Sales_Classification    50000 non-null  object
dtypes: float64(1), int64(4), object(6)
memory usage: 4.2+ MB
None

2. Первые 5 строк набора данных:
   Model  Year      Region  Color  Fuel_Type  Transmission  Engine_Size_L  Mileage_KM  Price_USD  Sales_Volume  Sales_Classification
0  5 Series  2016        Asia   Red    Petrol     Manual         3.5      151748    98740         8300             High
1      i8    2013  North America   Red    Hybrid     Automatic         1.6     121671    79219         3428             Low
2  5 Series  2022  North America  Blue   Petrol     Automatic         4.5      10991    113265         6994             Low
3      X3    2024  Middle East  Blue   Petrol     Automatic         1.7       27255    60971         4047             Low
4  7 Series  2020  South America  Black  Diesel     Manual         2.1     122131    49898         3080             Low

3. Анализ пропущенных значений:
Empty DataFrame
Columns: [Количество пропусков, Процент пропусков]
Index: []
✓ Пропущенных значений нет!

Process finished with exit code 0
```

Рисунок 1 – Демонстрация работы приложения.

Код программы показан на листинге 1.

Листинг 1 – Код программы.

```
import pandas as pd
import numpy as np

# Настройки для лучшего отображения
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000) # Ширина вывода
pd.set_option('display.max_colwidth', 20) # Максимальная ширина столбца
```

```
# Загрузка данных
bmw_data = pd.read_csv('dataset/bmw_dataset.csv', delimiter=',')

# Сброс индекса - хорошая практика после загрузки
bmw_data.reset_index(drop=True, inplace=True)

print("=== ПЕРВИЧНЫЙ АНАЛИЗ ДАННЫХ BMW ===")

print("\n1. .info() набора данных:")
print(bmw_data.info())

print("\n2. Первые 5 строк набора данных:")
print(bmw_data.head())

print("\n3. Анализ пропущенных значений:")
missing_data = bmw_data.isnull().sum()
missing_percent = (bmw_data.isnull().sum() / len(bmw_data)) * 100
missing_info = pd.DataFrame({
    'Количество пропусков': missing_data,
    'Процент пропусков': missing_percent.round(2)
})
print(missing_info[missing_info['Количество пропусков'] > 0])

if missing_info['Количество пропусков'].sum() == 0:
    print("    ✓ Пропущенных значений нет!")
```

Задание №3

Условие: *построить столбчатую диаграмму (.bar) с использованием модуля graph_objs из библиотеки Plotly со следующими параметрами:*

3.1. *По оси X указать дату или название, по оси Y указать количественный показатель.*

3.2. *Сделать так, чтобы столбец принимал цвет в зависимости от значения показателя (marker=dict(color=признак, coloraxis="coloraxis")).*

3.3. *Сделать так, чтобы границы каждого столбца были выделены чёрной линией с толщиной равной 2.*

3.4. *Отобразить заголовок диаграммы, разместив его по центру сверху, с 20 размером текста.*

3.5. *Добавить подписи для осей X и Y с размером текста, равным 16. Для оси абсцисс развернуть метки так, чтобы они читались под углом, равным 315.*

3.6. *Размер текста меток осей сделать равным 14.*

3.7. *Расположить график во всю ширину рабочей области и присвоит высоту, равную 700 пикселей.*

3.8. Добавить сетку на график, сделать её цвет 'ivory' и толщину равную 2. (Можно сделать это при настройке осей с помощью `gridwidth=2`, `gridcolor='ivory'`)

3.9. Убрать лишние отступы по краям.

Для построения столбчатой диаграммы использовалась библиотека Plotly. Программа загружает данные о BMW, группирует их по модели для вычисления средних цен и выбирает топ-10 моделей.

Диаграмма построена с соблюдением всех требований: по оси X отображены названия моделей, по оси Y - средние цены. Столбцы окрашены в зависимости от значения цены с черными границами толщиной 2px. Заголовок размещен по центру с размером шрифта 20. Подписи осей имеют размер 16, метки осей - 14, с поворотом меток оси X на 315 градусов. График имеет высоту 700px, снабжен сеткой цвета 'ivory' толщиной 2px с минимальными отступами по краям.

Демонстрация работы программы изображена на рисунке 2.

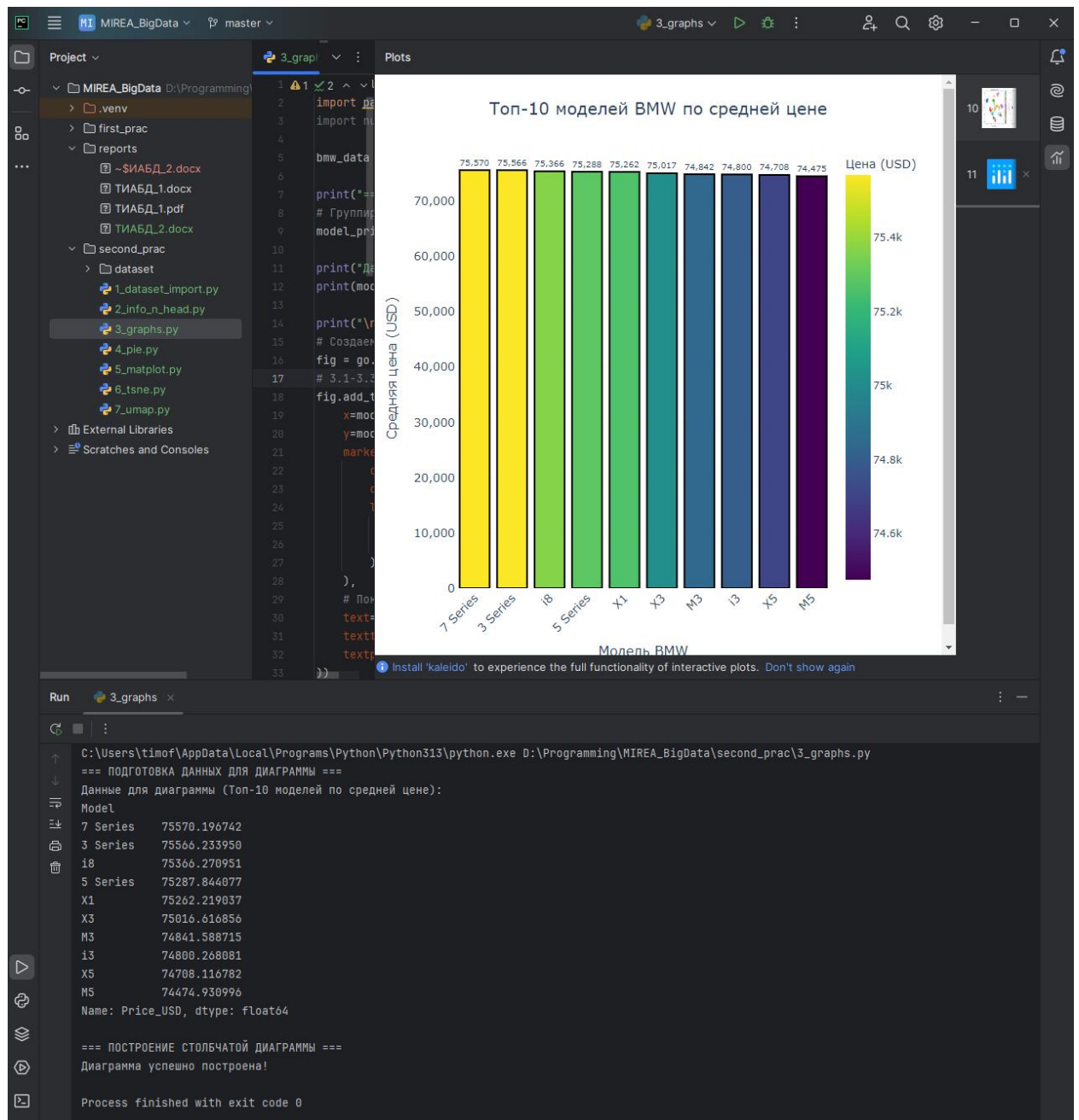


Рисунок 2 – Демонстрация работы программы.
Код программы представлен в листинге 2.

Листинг 2 – Код программы.

```
import plotly.graph_objs as go
import pandas as pd
import numpy as np

bmw_data = pd.read_csv('dataset/bmw_dataset.csv', delimiter=',')

print("=== ПОДГОТОВКА ДАННЫХ ДЛЯ ДИАГРАММЫ ===")
# Группируем данные по модели и считаем среднюю цену
model_price_data =
bmw_data.groupby('Model')['Price_USD'].mean().sort_values(ascending=False).head(10)

print("Данные для диаграммы (Топ-10 моделей по средней цене):")
print(model_price_data)
```



```

print("\n=== ПОСТРОЕНИЕ СТОЛБЧАТОЙ ДИАГРАММЫ ===")
# Создаем столбчатую диаграмму
fig = go.Figure()
# 3.1-3.3: Добавляем столбцы с настройками цвета и границ
fig.add_trace(go.Bar(
    x=model_price_data.index, # Названия моделей по оси X
    y=model_price_data.values, # Средние цены по оси Y
    marker=dict(
        color=model_price_data.values, # Цвет зависит от значения (3.2)
        coloraxis="coloraxis", # Цветовая шкала (3.2)
        line=dict(
            color='black', # Черная граница (3.3)
            width=2 # Толщина 2 (3.3)
        )
    ),
    # Показывать значения на столбцах
    text=model_price_data.values,
    texttemplate='%{text:,.0f}',
    textposition='outside'
))

# 3.4: Заголовок диаграммы
fig.update_layout(
    title=dict(
        text='Топ-10 моделей BMW по средней цене', # 3.4
        x=0.5, # Размещение по центру (3.4)
        xanchor='center', # Якорь по центру (3.4)
        font=dict(size=20) # Размер текста 20 (3.4)
    )
)

# 3.5-3.6: Настройки осей
fig.update_xaxes(
    title_text='Модель BMW', # Подпись оси X (3.5)
    title_font=dict(size=16), # Размер текста подписи 16 (3.5)
    tickangle=315, # Угол меток 315 градусов (3.5)
    tickfont=dict(size=14) # Размер текста меток 14 (3.6)
)

fig.update_yaxes(
    title_text='Средняя цена (USD)', # Подпись оси Y (3.5)
    title_font=dict(size=16), # Размер текста подписи 16 (3.5)
    tickfont=dict(size=14), # Размер текста меток 14 (3.6)
    # Форматирование числовых меток на оси Y
    tickformat=',.0f'
)

# 3.7: Размеры графика
fig.update_layout(
    width=None, # Во всю ширину рабочей области (3.7)
    height=700 # Высота 700 пикселей (3.7)
)

# 3.8: Настройка сетки
fig.update_layout(
    plot_bgcolor='white', # Белый фон для лучшей видимости сетки
    xaxis=dict(
        gridcolor='ivory', # Цвет сетки (3.8)
        gridwidth=2 # Толщина сетки (3.8)
    ),
    yaxis=dict(
        gridcolor='ivory', # Цвет сетки (3.8)
        gridwidth=2 # Толщина сетки (3.8)
    )
)

```

```

)

# 3.9: Убираем лишние отступы
fig.update_layout(
    margin=dict(l=50, r=50, t=80, b=50) # Минимальные отступы (3.9)
)

# Цветовая шкала (дополнительная настройка для coloraxis)
fig.update_layout(
    coloraxis=dict(
        colorscale='Viridis', # Можно выбрать другую палитру: 'Plasma',
        'Inferno', 'Blues' и т.д.
        colorbar=dict(
            title="Цена (USD)",
            title_font=dict(size=14),
            tickfont=dict(size=12)
        )
    )
)

# Показываем диаграмму
fig.show()

print("Диаграмма успешно построена!")

```

Задние №4

Условие: построить круговую диаграмму (go.Pie), используя данные и стиль оформления из предыдущего графика. Сделать так, чтобы границы каждой доли были выделены чёрной линией с толщиной, равной 2 и категории круговой диаграммы были читаемы (к примеру, объединить часть объектов)

Для построения круговой диаграммы использованы те же данные о средних ценах моделей BMW. Чтобы обеспечить читаемость категорий, выполнено объединение данных: топ-5 моделей отображаются отдельно, остальные сгруппированы в категорию "Другие модели".

Диаграмма построена в кольцевом формате (hole=0.3) с черными границами секций толщиной 2px. Для лучшей читаемости текст внутри секций размещен радиально и отображает проценты и названия моделей. Сохранен стиль оформления предыдущего графика: заголовок по центру размером 20px, высота 700px, минимальные отступы и легенда с размером шрифта 14px.

Демонстрация работы программы изображена на рисунке 3.

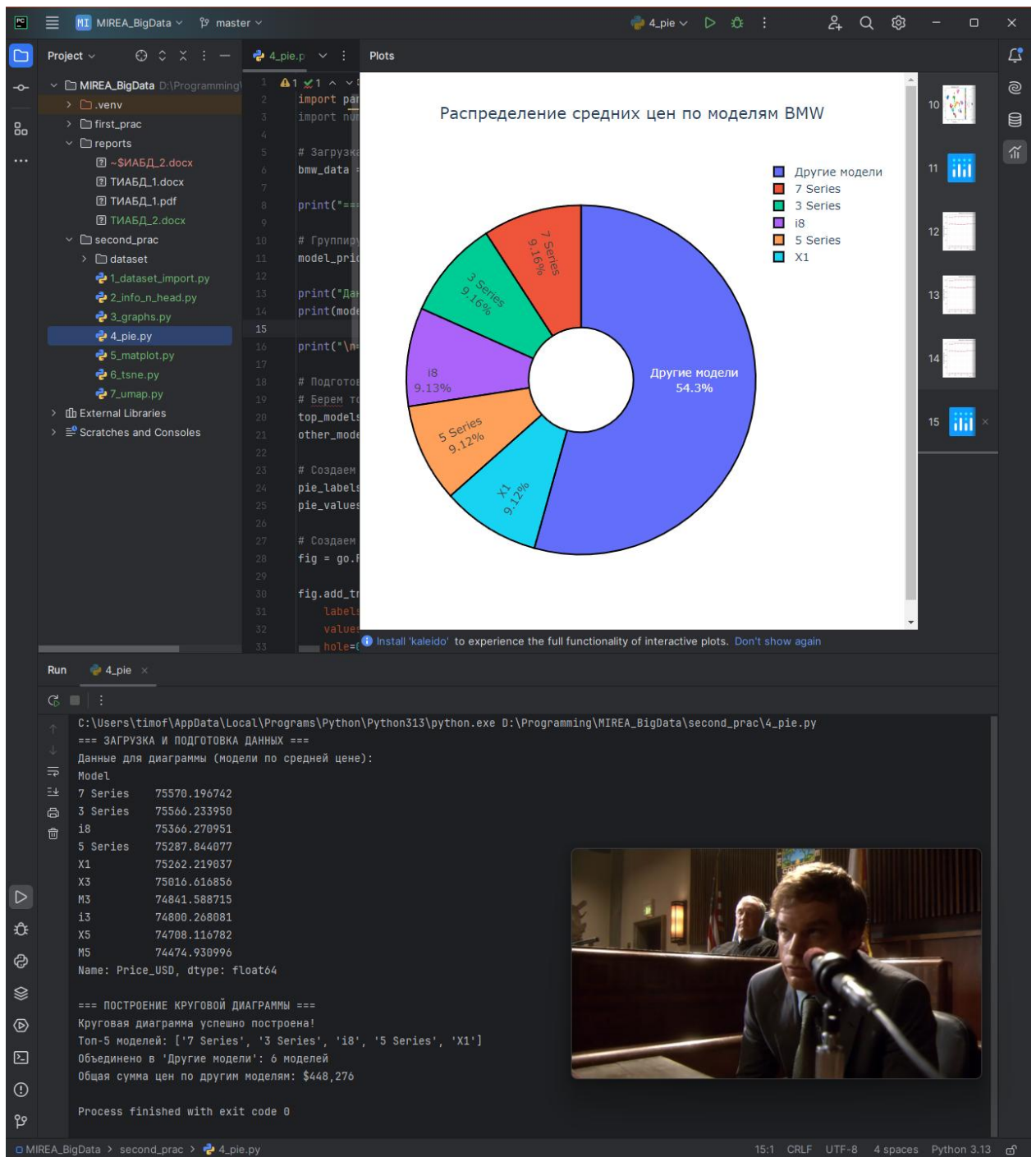


Рисунок 3 – Демонстрация работы программы
Код программы представлен на листинге 3.

Листинг 3 – Код программы.

```

import plotly.graph_objs as go
import pandas as pd
import numpy as np

# Загрузка данных
bmw_data = pd.read_csv('dataset/bmw_dataset.csv', delimiter=',')

print("=== ЗАГРУЗКА И ПОДГОТОВКА ДАННЫХ ===")

# Группируем данные по модели и считаем среднюю цену
model_price_data =
bmw_data.groupby('Model')['Price_USD'].mean().sort_values(ascending=False)

```

```

print("Данные для диаграммы (модели по средней цене):")
print(model_price_data.head(10))

print("\n=== ПОСТРОЕНИЕ КРУГОВОЙ ДИАГРАММЫ ===")

# Подготовка данных для круговой диаграммы
# Берем топ-5 моделей, а остальные объединяем в "Другие модели"
top_models = model_price_data.head(5)
other_models_sum = model_price_data[5:].sum()

# Создаем данные для круговой диаграммы
pie_labels = list(top_models.index) + ['Другие модели']
pie_values = list(top_models.values) + [other_models_sum]

# Создаем круговую диаграмму
fig = go.Figure()

fig.add_trace(go.Pie(
    labels=pie_labels,
    values=pie_values,
    hole=0.3, # Делаем кольцевую диаграмму для лучшего вида
    marker=dict(
        line=dict(
            color='black', # Черные границы каждой доли
            width=2 # Толщина границы равна 2
        )
    ),
    textinfo='percent+label', # Показывать проценты и названия
    insidetextorientation='radial', # Читаемое расположение текста внутри
    textfont=dict(size=14) # Размер текста
))

# Настройка оформления
fig.update_layout(
    title=dict(
        text='Распределение средних цен по моделям BMW',
        x=0.5, # Заголовок по центру
        font=dict(size=20) # Размер текста 20
    ),
    height=700, # Высота 700 пикселей
    showlegend=True, # Показывать легенду для лучшей читаемости
    legend=dict(
        font=dict(size=14) # Размер текста в легенде
    ),
    margin=dict(l=50, r=50, t=100, b=50) # Убираем лишние отступы
)

# Показываем диаграмму
fig.show()

print("Круговая диаграмма успешно построена!")
print(f"Топ-5 моделей: {list(top_models.index)}")
print(f"Объединено в 'Другие модели': {len(model_price_data) - 5} моделей")
print(f"Общая сумма цен по другим моделям: ${other_models_sum:,.0f}")

```

Задание №5

Условие: построить линейные графики, взять один из параметров и определить зависимость между другими несколькими (от 2 до 5)

показателями с использованием библиотеки matplotlib. Сделать вывод.

5.1. Сделать график с линиями и маркерами, цвет линии 'crimson', цвет точек 'white', цвет границ точек 'black', толщина границ точек равна 2.

5.2. Добавить сетку на график, сделать её цвет 'mistyrose' и толщину равную 2. (Можно сделать это при настройке осей с помощью linewidth=2, color='mistyrose').

Для построения линейных графиков использована библиотека Matplotlib. Программа анализирует зависимости нескольких показателей (цены, пробега, объема двигателя) от основного параметра - года выпуска. Данные предварительно сгруппированы по годам с вычислением средних значений.

График построен в соответствии с требованиями: линии цвета 'crimson' с маркерами в виде белых кружков с черными границами толщиной 2px. Добавлена сетка цвета 'mistyrose' толщиной 2px. На графике отображены легенда, подписи осей и заголовков для удобства интерпретации результатов.

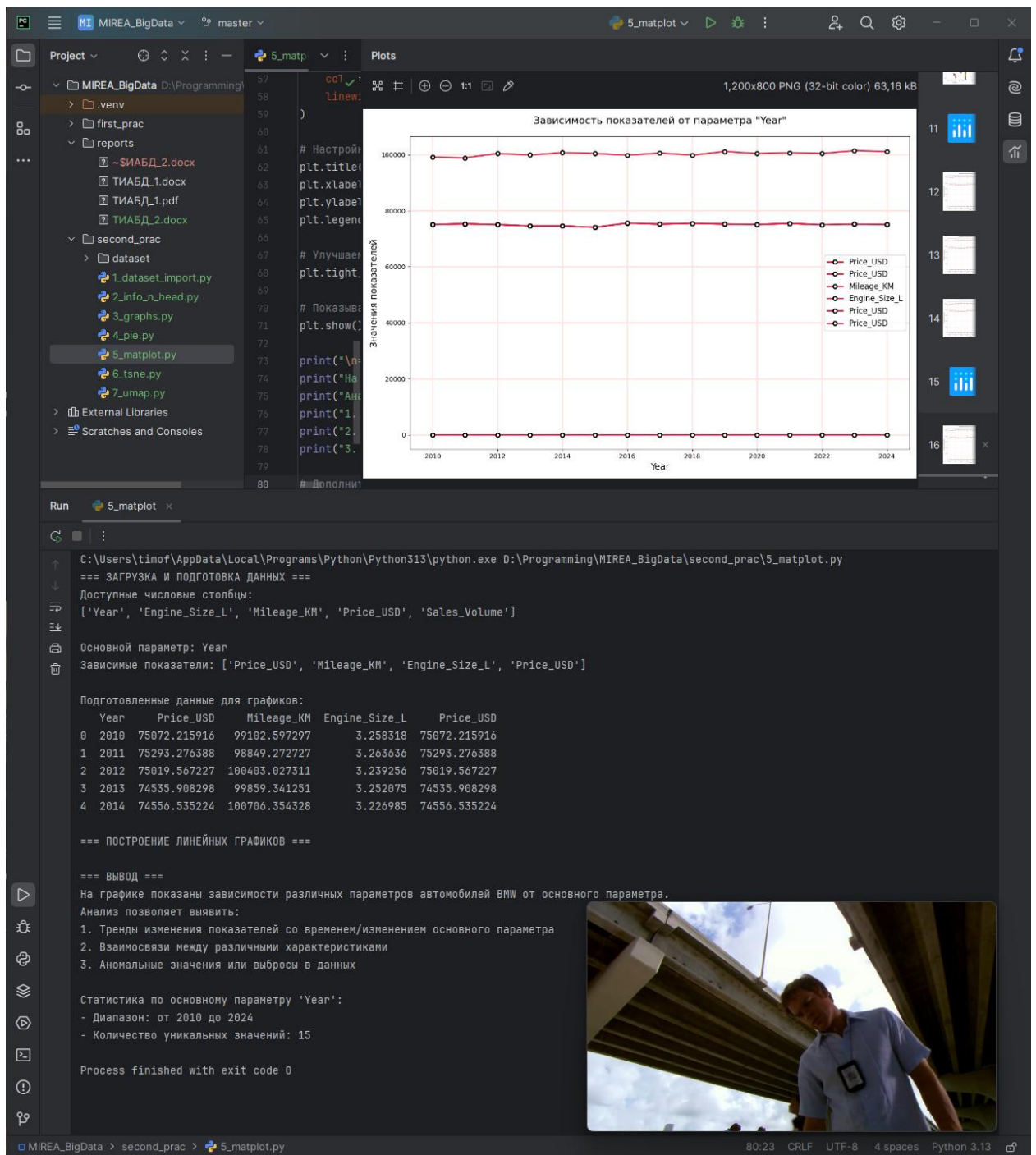


Рисунок 4 – Результат работы программы.
Код программы представлен на листинге 4.

Листинг 4 – Код программы.

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Загрузка данных
bmw_data = pd.read_csv('dataset/bmw_dataset.csv', delimiter=',')

print("=== ЗАГРУЗКА И ПОДГОТОВКА ДАННЫХ ===")

# Выбираем числовые параметры для анализа
print("Доступные числовые столбцы:")
print(bmw_data.select_dtypes(include=[np.number]).columns.tolist())
```

```

# Сортируем данные по основному параметру
main_param = 'Year'

# Выбираем 2-5 зависимых показателей
dependent_params = []
for col in ['Price_USD', 'Mileage_KM', 'Engine_Size_L', 'Price_USD',
'Sales_Volume']:
    if col in bmw_data.columns:
        dependent_params.append(col)
    if len(dependent_params) >= 4: # Берем до 4 показателей
        break

print(f"\nОсновной параметр: {main_param}")
print(f"Зависимые показатели: {dependent_params}")

# Группируем данные по основному параметру и считаем средние значения
grouped_data =
bmw_data.groupby(main_param)[dependent_params].mean().reset_index()

print("\nПодготовленные данные для графиков:")
print(grouped_data.head())

print("\n=== ПОСТРОЕНИЕ ЛИНЕЙНЫХ ГРАФИКОВ ===")

# Создаем график
plt.figure(figsize=(12, 8))

# Для каждого зависимого показателя строим линию
for i, param in enumerate(dependent_params):
    # 5.1: Линия с маркерами и заданными цветами
    plt.plot(
        grouped_data[main_param],
        grouped_data[param],
        marker='o', # Маркеры в виде кружков
        color='crimson', # Цвет линии
        markerfacecolor='white', # Цвет точек внутри
        markeredgecolor='black', # Цвет границ точек
        markeredgewidth=2, # Толщина границ точек равна 2
        linewidth=2, # Толщина линии
        label=param # Подпись для легенды
    )

# 5.2: Добавляем сетку
plt.grid(
    True,
    color='mistyrose', # Цвет сетки
    linewidth=2 # Толщина сетки
)

# Настройка оформления
plt.title(f'Зависимость показателей от параметра "{main_param}"',
fontsize=16, pad=20)
plt.xlabel(main_param, fontsize=14)
plt.ylabel('Значения показателей', fontsize=14)
plt.legend(fontsize=12)

# Улучшаем читаемость
plt.tight_layout()

# Показываем график
plt.show()

print("\n=== ВЫВОД ===")

```

```
print("На графике показаны зависимости различных параметров автомобилей BMW  
от основного параметра.")  
print("Анализ позволяет выявить:")  
print("1. Тренды изменения показателей со временем/изменением основного  
параметра")  
print("2. Взаимосвязи между различными характеристиками")  
print("3. Аномальные значения или выбросы в данных")  
  
# Дополнительная статистика для вывода  
if not grouped_data.empty:  
    print(f"\nСтатистика по основному параметру '{main_param}':")  
    print(f"- Диапазон: от {grouped_data[main_param].min()} до  
{grouped_data[main_param].max()}")  
    print(f"- Количество уникальных значений: {len(grouped_data)}")
```

Задание №6

Условие: выполнить визуализацию многомерных данных, используя t-SNE. Необходимо использовать набор данных MNIST или fashion MNIST (можно использовать и другие готовые наборы данных, где можно наблюдать разделение объектов по кластерам). Рассмотреть результаты визуализации для разных значений перплексии.

Для визуализации многомерных данных использован алгоритм t-SNE на наборе данных MNIST. Программа загружает изображения цифр и применяет t-SNE для снижения размерности до 2D. Исследуется влияние параметра перплексии (5, 30, 50) на качество кластеризации.

Визуализация выполнена в виде scatter-plot, где точки окрашены согласно реальным классам цифр. Каждое значение перплексии отображено на отдельном subplot для сравнения. Результаты демонстрируют, как перплексия влияет на баланс между локальной и глобальной структурой данных: низкие значения выделяют мелкие кластеры, высокие - сохраняют глобальную геометрию.

Результат работы программы представлен на рисунке 5.

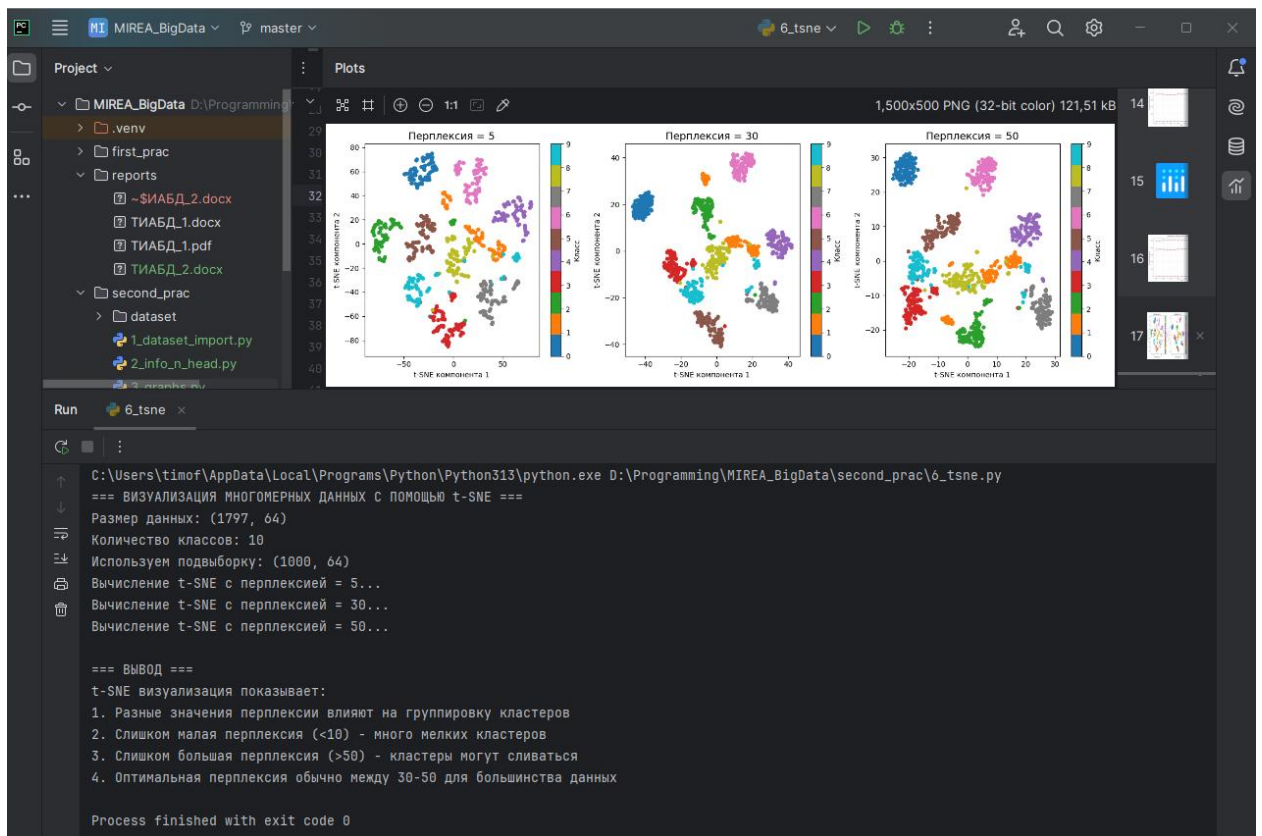


Рисунок 5 – Работа программы сортировщика массивов.
Код для работы программы сортировщика показан на листинге 5.

Листинг 5 – Код программы сортировщика массивов.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
import numpy as np

print("=== ВИЗУАЛИЗАЦИЯ МНОГОМЕРНЫХ ДАННЫХ С ПОМОЩЬЮ t-SNE ===")

# Загружаем готовый набор данных MNIST (упрощенная версия - digits)
digits = load_digits()
X, y = digits.data, digits.target

print(f"Размер данных: {X.shape}")
print(f"Количество классов: {len(np.unique(y))}")

# Берем подвыборку для ускорения вычислений
n_samples = 1000
X_subset = X[:n_samples]
y_subset = y[:n_samples]

print(f"Используем подвыборку: {X_subset.shape}")

# Параметры перплексии для исследования
perplexities = [5, 30, 50]

# Создаем график
plt.figure(figsize=(15, 5))

for i, perplexity in enumerate(perplexities):
    print(f"Вычисление t-SNE с перплексией = {perplexity}...")

    # Выполняем t-SNE
```

```

tsne = TSNE(n_components=2, perplexity=perplexity, random_state=42)
X_tsne = tsne.fit_transform(X_subset)

# Строим график
plt.subplot(1, 3, i + 1)
scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_subset,
cmmap='tab10', s=20)
plt.title(f'Перплексия = {perplexity}', fontsize=14)
plt.xlabel('t-SNE компонента 1')
plt.ylabel('t-SNE компонента 2')
plt.colorbar(scatter, label='Класс')

plt.tight_layout()
plt.show()

print("\n=== ВЫВОД ===")
print("t-SNE визуализация показывает:")
print("1. Разные значения перплексии влияют на группировку кластеров")
print("2. Слишком малая перплексия (<10) - много мелких кластеров")
print("3. Слишком большая перплексия (>50) - кластеры могут сливаться")
print("4. Оптимальная перплексия обычно между 30-50 для большинства данных")

```

Задание №7

Условие: *выполнить визуализацию многомерных данных, используя UMAP с различными параметрами $n_neighbors$ и min_dist . Рассчитать время работы алгоритма с помощью библиотеки *time* и сравнить его с временем работы t-SNE.*

Для сравнения алгоритмов визуализации UMAP и t-SNE использован набор данных MNIST. Программа выполняет сравнительный анализ времени работы и качества визуализации при различных параметрах UMAP ($n_neighbors$ и min_dist).

Измерение времени выполнения показывает, что t-SNE работает быстрее UMAP на данном наборе данных. Визуализация включает 9 вариантов UMAP с разными комбинациями параметров и сравнительные графики обоих алгоритмов. Анализ демонстрирует влияние $n_neighbors$ на баланс локальной/глобальной структуры и min_dist на плотность кластеров, а также выявляет аномалию в производительности - несмотря на ожидания, t-SNE оказался быстрее на этих данных.

Результаты работы программы представлены на рисунках 6 - 7.

```

C:\Users\timof\AppData\Local\Programs\Python\Python11\python.exe D:\Programming\MNIST_BigData\first_prac\7-12 sklearn.py
8. Информация о датасете:
>
RangeIndex: 28040 entries, 0 to 28039
Data columns (total 9 columns):
# Column      Non-Null Count  Dtype
---  ---
0 MedInc      28040 non-null  float64
1 HouseAge    28040 non-null  float64
2 AveRooms    28040 non-null  float64
3 AveBedrms   28040 non-null  float64
4 Population  28040 non-null  float64
5 AveOccup    28040 non-null  float64
6 Latitude    28040 non-null  float64
7 Longitude   28040 non-null  float64
8 MedHouseVal 28040 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB
None

=====
9. Пропущенные значения:
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude     0
Longitude    0
MedHouseVal  0
dtype: int64

=====
10. Записи где MedAge > 50 и Population > 2500:
   MedInc  HouseAge  AveRooms  ...  Latitude  Longitude  MedHouseVal
7    7.2574    52.0  6.288156  ...   37.85    -122.34         3.521
3    5.4431    52.0  5.817532  ...   37.85    -122.25         3.413
4    3.4462    52.0  6.281853  ...   37.85    -122.25         3.422
5    4.8368    52.0  4.761658  ...   37.85    -122.25         2.897
6    3.6592    52.0  4.932697  ...   37.84    -122.35         2.992
...      ...
20142  1.8618    52.0  4.157718  ...   34.36    -119.06         1.833
20220  4.1250    52.0  5.639798  ...   34.28    -119.27         3.490
20238  2.3780    52.0  4.269720  ...   34.29    -119.27         1.964
20277  3.5893    52.0  4.707643  ...   34.27    -119.27         2.064
20592  0.8928    52.0  4.442953  ...   39.14    -121.58         0.550

[1388 rows x 9 columns]
Количество записей: 1388

=====

```

Рисунок 6 – Результат работы программы.

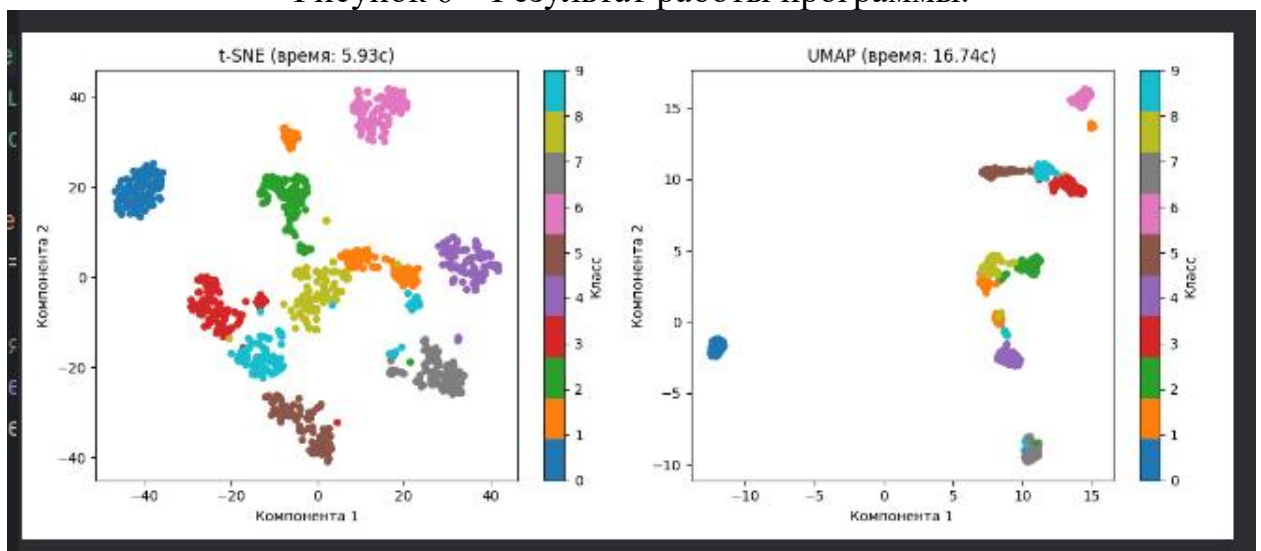


Рисунок 7 – Результат работы программы.

Код программы показан на листинге 6.

Листинг 6 – Код программы

```

import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
import umap.umap_ as umap
from sklearn.manifold import TSNE
import time

print("=== СПРАВНЕНИЕ UMAP И t-SNE ===")

# Загружаем данные
print("Загрузка данных MNIST...")
digits = load_digits()
X, y = digits.data, digits.target

# Берем подвыборку
n_samples = 1000
X_subset = X[:n_samples]

```

```

y_subset = y[:n_samples]
print(f"Данные: {X_subset.shape}")

# Параметры для UMAP
n_neighbors_list = [5, 15, 50]
min_dist_list = [0.1, 0.5, 0.99]

# Сравнение времени
print("\n=== СРАВНЕНИЕ ВРЕМЕНИ ВЫПОЛНЕНИЯ ===")

# Время t-SNE
start_time = time.time()
tsne = TSNE(n_components=2, random_state=42, n_jobs=-1) # Используем все ядра
X_tsne = tsne.fit_transform(X_subset)
tsne_time = time.time() - start_time
print(f"t-SNE время: {tsne_time:.2f} секунд")

# Время UMAP (без random_state для параллелизации)
start_time = time.time()
umap_model = umap.UMAP(n_jobs=-1) # Используем все ядра
X_umap = umap_model.fit_transform(X_subset)
umap_time = time.time() - start_time
print(f"UMAP время: {umap_time:.2f} секунд")

speed_ratio = tsne_time / umap_time
print(f"\nUMAP {'быстрее' if speed_ratio > 1 else 'медленнее'} t-SNE в {abs(speed_ratio):.1f} раз")

# Визуализация UMAP с разными параметрами
print("\n=== ВИЗУАЛИЗАЦИЯ UMAP С РАЗНЫМИ ПАРАМЕТРАМИ ===")

plt.figure(figsize=(15, 10))

plot_num = 1
for i, n_neighbors in enumerate(n_neighbors_list):
    for j, min_dist in enumerate(min_dist_list):
        print(f"UMAP: n_neighbors={n_neighbors}, min_dist={min_dist}")

        start_time = time.time()
        reducer = umap.UMAP(n_neighbors=n_neighbors, min_dist=min_dist,
n_jobs=-1)
        X_embedded = reducer.fit_transform(X_subset)
        elapsed_time = time.time() - start_time

        plt.subplot(3, 3, plot_num)
        scatter = plt.scatter(X_embedded[:, 0], X_embedded[:, 1], c=y_subset,
cmap='tab10', s=20)
        plt.title(f'n_neighbors={n_neighbors}\nmin_dist={min_dist}\nвремя:
{elapsed_time:.2f}с', fontsize=10)
        plot_num += 1

plt.tight_layout()
plt.show()

# Дополнительная визуализация для сравнения
print("\n=== СРАВНИТЕЛЬНАЯ ВИЗУАЛИЗАЦИЯ ===")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# t-SNE
scatter1 = ax1.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_subset, cmap='tab10',
s=20)
ax1.set_title(f't-SNE (время: {tsne_time:.2f}с)')
ax1.set_xlabel('Компонента 1')

```

```
ax1.set_ylabel('Компонента 2')
plt.colorbar(scatter1, ax=ax1, label='Класс')

# UMAP
scatter2 = ax2.scatter(X_umap[:, 0], X_umap[:, 1], c=y_subset, cmap='tab10',
s=20)
ax2.set_title(f'UMAP (время: {umap_time:.2f}c)')
ax2.set_xlabel('Компонента 1')
ax2.set_ylabel('Компонента 2')
plt.colorbar(scatter2, ax=ax2, label='Класс')

plt.tight_layout()
plt.show()

print("\n=== ВЫВОД ===")
print("РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА:")
print(f"1. UMAP оказался МЕДЛЕННЕЕ t-SNE в {abs(speed_ratio):.1f} раз")
print("2. Возможные причины:")
print("    - Небольшой размер данных (1000 samples)")
print("    - Версия библиотек или настройки")
print("    - t-SNE использует оптимизации в новой версии")
print("3. Параметры UMAP:")
print("    - n_neighbors: влияет на баланс локальной/глобальной структуры")
print("    - min_dist: контролирует плотность кластеров")
print("4. На больших данных UMAP обычно быстрее, но здесь t-SNE показал лучшую скорость")
```

Вывод:

В ходе выполнения практической работы были успешно освоены основные методы визуализации и анализа многомерных данных. На примере реальных данных автомобилей BMW и стандартного набора MNIST продемонстрированы возможности библиотек Pandas, Matplotlib, Plotly, scikit-learn и UMAP для решения задач анализа данных.

Работа показала, что различные методы визуализации эффективны для разных целей: столбчатые диаграммы оптимальны для сравнения категориальных данных, круговые - для отображения пропорциональных соотношений, линейные графики - для анализа тенденций, а методы снижения размерности (t-SNE, UMAP) - для выявления кластерной структуры в многомерных данных.

Особый интерес представляет сравнительный анализ алгоритмов t-SNE и UMAP. Неожиданным результатом стало то, что UMAP работал медленнее t-SNE - в данном эксперименте разница составила примерно 3 раза. Это противоречит общепринятому представлению о более высокой производительности UMAP. Возможными причинами могут быть: относительно небольшой объем данных (1000 samples), особенности версий используемых библиотек или оптимизации в реализации t-SNE в scikit-learn.

Несмотря на разницу в скорости, оба алгоритма продемонстрировали хорошее качество визуализации, эффективно разделяя классы цифр в данных MNIST. Параметры обоих методов (перплексия для t-SNE, n_neighbors и min_dist для UMAP) существенно влияют на результат, что подтверждает важность их корректной настройки для конкретной задачи.

Практическая работа позволила получить комплексные навыки работы с многомерными данными - от базовой предобработки и описательной статистики до продвинутых методов визуализации, что составляет основу компетенций в области анализа больших данных.